EVAT Site Suitability Analysis API Technical Documentation

EV Charging Station Location Prioritization Pipeline

Submitted to

Deakin University

22nd of March 2026

by

Hetti Arachchige Amantha Kalhara Kulathunga

Chameleon

Smarter World

1. Executive Summary

This document provides full technical documentation for the EVAT Site Suitability API, which is a FastAPI based web service that indicates the results of a machine learning pipeline designed to identify and rank the most suitable locations for new electric vehicle (EV) charging stations across Victoria, Australia. The API was implemented as a part of the Chameleon EVAT project at Deakin University. Furthermore, this API allows developers and planners to query suitability scores by suburb name (SA2 area), GPS coordinates, or retrieve a ranked list of the top candidate sites. The underlying model uses a composite scoring method that incorporates traffic volume, amenity density, and distance to existing chargers, weighted and normalized to produce a single suitability score per location.

Key capabilities

- Query EV charger site suitability by SA2 area name
- Query suitability by latitude and longitude coordinates
- Retrieve a ranked list of the top N most suitable sites
- Returns cluster labels indicating the demand/ supply profile of each area
- Built on FastAPI with auto generated Swagger documentation

2. Data Sources

The pipeline combines several public and government datasets to build a comprehensive spatial profile of each SA2 area in Victoria. These datasets were selected because they are publicly accessible and cover the key dimensions of EV charging demand and supply.

Dataset	Source	Columns Added to Final Dataset
SA2 Boundaries	Australian Bureau of Statistics (ABS) — SA2 2021 Shapefile	SA2_CODE21, SA2_NAME21, geometry_x
Population Data	Australian Bureau of Statistics (ABS) – Population dataset at the SA2 (2023–2024)	Population density 2024 (persons/km2), Total Population
IRSAD Score	Australian Bureau of Statistics (ABS) – The Socio-Economic Indexes for Areas (SEIFA) 2021	IRSAD Score
EV Charger Locations	TomTom Places API – Charging station locations across Victoria	ChargerCount
Traffic Volume	VicRoads (Source not explicitly mentioned; inferred by AADT data) — Traffic Flow in Victoria (Traffic_flow_in_victoria_new.geojson)	Average Annual Daily Traffic Volume
Amenity Data	OpenStreetMap (Source not explicitly mentioned; inferred from filename) — Points of Interest (gis_osm_pois_free_1.shp)	AmenitiesCount

All feature engineering and dataset merging at the SA2 level was performed in the data preprocessing notebook, which combined these sources into a single unified dataset used as input to the model.

3. Technical Dependencies

Required Python Libraries

The following libraries are required at each stage

Stage	Libraries
Preprocessing	geopandas, shapely, json, requestgs, time, folium, branca
Clustering	Sklearn.preprocessing.StandardScaler, sklearn.cluster.KMeans, sklearn.metrics.silhouette_score, seaborn
Scoring	Sklearn.preprocessing.MinMaxScaler, shapely.ops.unary_union, folium, matplotlib
API	fastapi, uvicorn, pandas, geodesic

Installation

- To run the API, import the following libraries:
  fastapi, uvicorn, pandas, geopy

- To run the full pipeline including preprocessing, clustering and scoring, import the additional stage specific dependencies

Additional Notes

- The preprocessing and scoring notebooks were developed in Google Colab, hence if running locally, ensure geopandas is installed in a compatible environment; recommended to use conda

- uvicorn is not imported in main.py but it is required to start the API server from the terminal

4. Model Architecture

Pipeline overview

The system operates as a four-stage data pipeline. Raw data from five government and public sources is collected, cleaned, and merged in the preprocessing stage. The merged dataset is then fed into a KMeans clustering algorithm to classify each SA2 area by its demand and supply characteristics. A separate suitability scoring model then ranks locations more precisely based on localized spatial factors. finally, the results are exposed through the FastAPI service so they can be consumed by the EVAT web application.

Stage	Notebook	Description	Output
1 — Data Collection & Preprocessing	project_par1_data_preprocessing.ipynb	Five datasets (SA2 boundaries, population, IRSAD, EV chargers, amenities, traffic) are merged into one master table with one row per SA2 area	final_data_sa2.csv — 522 rows with columns: SA2_CODE21, SA2_NAME21, Population density 2024(persons/km2), Total Population, IRSAD Score, ChargerCount, AmenitiesCount, Average Annual Daily Traffic Volume, geometry_x
2 — KMeans Clustering	Project_part_2_Final_part.ipynb	KMeans (k = 6) groups SA2 areas into demand/supply categories based on all 6 features to determine what type of area is this?	Cluster_Label column added to dataset; cluster map exported
3 — Suitability Scoring	Project_part_2_Final_part.ipynb & Suitability_Score_Model-3.ipynb	Candidate points are generated inside High Demand/Low Supply SA2s. Traffic, amenity, and charger distance features are computed per point, normalised, and combined into a weighted SiteScore. T3 refines this into NewSuitabilityScore	priority_charger_sites_top1.csv — top ranked candidate sites with scores, ranks, and coordinates
4 — API	main.py	FastAPI loads priority_sites.csv and exposes the results via 4 endpoints for use by the EVAT web application	JSON responses via GET endpoints

Why Different Feature Sets Across Stages?

Clustering (Stage 2) uses all six SA2-level features because it needs a broad demographic and infrastructure picture to meaningfully classify what type of area each suburb is. Population, socioeconomic status (IRSAD), and existing charger counts are all relevant to understanding area character. Thereafter, suitability scoring (Stage 3) narrows to three highly localized spatial features, such as traffic within 200m, amenities within 300m, and distance to nearest charger, because these most directly reflect where a charger would get the most practical use and fill the biggest infrastructure gap.

KMeans Clustering Method

KMeans clustering was applied in Project_part_2_Final_part.ipynb using the following six features after StandardScaler normalization:

- Population density 2024(persons/km2)
- Total Population
- IRSAD Score
- ChargerCount
- AmenitiesCount
- Average Annual Daily Traffic Volume.

The optimal number of clusters (k=6) was determined using the Elbow Method, with the Silhouette Score and Davies-Bouldin Index used to validate cluster quality.

Cluster Labels

The following six cluster labels were defined in Project_part_2_Final_part.ipynb and assigned to each SA2 area based on the KMeans output. These labels are returned by the API in every suitability response.

Cluster ID	Label	Description
0	High Demand, Low Supply	Priority areas — high EV need, few existing chargers. These SA2s are the focus of stage 3 candidate site generation
1	Urban Hotspots	Dense urban areas with moderate charger supply
2	Very Low Demand / Rural Transit	Rural areas with minimal demand

3	High-Traffic Corridors (Undersupplied)	Busy routes lacking charging infrastructure
4	Traffic Hotspots	High vehicle movement, varied supply
5	Well Supplied	Areas with adequate existing charger coverage

Suitability Scoring Method

Stage 3 generates candidate points inside High Demand, Low Supply SA2s (Cluster 0), within 500m of major roads and near at least one amenity. For each candidate point, three spatial features are computed and normalized using MinMaxScaler before being combined into a weighted composite score.

The scoring formula was first implemented in Project_part_2_Final_part.ipynb and refined in Suitability_Score_Model-3.ipynb as the NewSuitabilityScore. It is as follows:

```
SiteScore  =  (0.40 x Traffic_200m_normalised)
            + (0.30 x AmenityScore_300m_normalised)
            + (0.30 x dist_to_ch_m_normalised)
```

```
NewSuitabilityScore  = (0.40 x traffic_scaled)
                     + (0.35 x amenity_scaled)
                     + (0.25 x distance_scaled)
```

The following summarizes the weight and rationale carried by each feature used:

Feature	SiteScore (initial)	NewSuitabilityScore (revised)	Rationale
Traffic Volume (within 200m)	40%	40%	Higher traffic = more potential charger users

Amenity Score (within 300m)	30%	35%	Nearby amenities allow charging during errands
Distance to Nearest Charger	30%	25%	Greater distance = underserved area, higher priority

Saved Model Assets

The trained model and supporting files are available in the /Use_Cases/Site Suitability Analysis folder as follows:

- priority_charger_sites (2).html
- priority_charger_sites_pointwise(1).html
- priority_charger_sites_top1 (2).csv

However, Main.py requires the following file to execute, which is missing. priority_charger_sites_top1 (2). Csv is the closest example which can be amended to fit this file

File	Description
priority_sites.csv	Main dataset containing all SA2 areas with pre-computed suitability scores, cluster labels and the following columns: SA2_CODE21, SA2_NAME21, Population density 2024(persons/km2), Total Population, IRSAD Score, ChargerCount, AmenitiesCount, Average Annual Daily Traffic Volume, geometry_x, cluster_label, suitability_score, rank, lat, lon

5. API Endpoint Reference

The API is built using FastAPI and exposes four endpoints. All endpoints use the HTTP GET method and return JSON responses. The implementation is defined in main.py.

5.1 GET /

Health check endpoint. Confirms the API service is running.

- Request: GET http://127.0.0.1:8000/
- Parameters: None
- Response: { "message": "Site Suitability API is running!" }

5.2 GET /suitability/area/{sa2_name}

Returns the full suitability profile for a specific SA2 area, looked up by name. The name match is case-insensitive.

- Parameters:

Parameter	Type	Location	Required	Description
sa2_name	string	Path	Yes	Name of the SA2 area

- Example Request: GET http://127.0.0.1:8000/suitability/area/Wallan
- Example Response:


```
{
  "sa2": "Wallan",
  "cluster_label": "High Demand, Low Supply",
  "population": 28431.2,
  "chargers": 3,
  "avg_daily_traffic": 9785.195,
  "amenity_score": 0.72,
  "irsad_score": 1004,
  "suitability_score": 0.81,
  "rank": 12
}
```


- Error Response: HTTP 404: { "detail": "SA2 area not found." }

5.3 GET /suitability/top

Returns the top N SA2 areas ranked by suitability score, from highest to lowest. The default value of N is 10 if no parameter is supplied

- Parameters:

Parameter	Type	Location	Required	Default	Description
n	integer	Query	No	10	Number of top sites to return

- Example Request: GET http://127.0.0.1:8000/suitability/top?n=5

- Example Response:


```
[
  { "sa2": "Melbourne CBD - North", "rank": 1, "suitability_score": 0.94 },
  { "sa2": "Southbank - East",      "rank": 2, "suitability_score": 0.91 },
  { "sa2": "Footscray",              "rank": 3, "suitability_score": 0.88 },
  { "sa2": "Richmond - South",      "rank": 4, "suitability_score": 0.86 },
  { "sa2": "Dandenong",              "rank": 5, "suitability_score": 0.83 }
]
```

5.4 GET /suitability/coords

Finds the nearest SA2 area to a given GPS coordinate and returns its suitability profile. Uses geodesic (great-circle) distance calculations via the geopy library.

- Parameters:

Parameter	Type	Location	Required	Description
lat	float	Query	Yes	Latitude of the location (decimal degrees)
lng	float	Query	Yes	Longitude of the Location (decimal degrees)

- Example Request: GET http://127.0.0.1:8000/suitability/coords?lat=-37.8136&lng=144.9631

- Example Response:


```
{
  "sa2": "Melbourne CBD - North",
  "distance_km": 0.43,
  "suitability_score": 0.94,
  "cluster_label": "Urban Hotspots",
  "rank": 1
}
```

6. Setup and Running the API

The following steps describe how to run the Site Suitability API locally on your machine

- **Step 1 — Clone the Repository**

  git clone https://github.com/Chameleon-company/EVAT-Data-Science.git

  cd EVAT-Data-Science/Use_Cases/Site\ Suitability\ Analysis

- **Step 2 — Install Dependencies**

  Install all required libraries for running main.py

  Pip install fastai uvicorn pandas geopy (the original requirements.txt is incomplete)

- **Step 3 — Ensure Data File is Present**

  The file priority_sites.csv must be present in the same directory as main.py. This file contains all pre-computed suitability scores, cluster labels, and geographic data for every SA2 area in Victoria.

- **Step 4 — Run the API**

  Start the API server using uvicorn

  uvicorn main:app --reload

- **Step 5 — Access the Interactive Documentation**

  Once running, open your browser and navigate to: http://127.0.0.1:8000/docs

  FastAPI automatically generates an interactive Swagger UI page where all endpoints can be tested directly in the browser without needing any additional tools.

7. Limitations and Assumptions

Limitations

The following limitations were identified based on the current state of the implementation and were noted in the project progress documentation from T3, 2025.

Limitation	Detail
Local deployment only	The API has been tested locally and is not yet deployed to a shared or live server environment.
No frontend integration	There is currently no connection between the API outputs and the EVAT web or mobile application interface.
Static dataset	The API reads from a pre-computed CSV file and does not update dynamically as new chargers are installed or traffic data changes (Note: this CSV file is missing).
Victoria only	The model and dataset cover Victorian SA2 areas only. Other Australian states are not supported.

Assumptions

The model is built on the following assumptions

Assumption	Detail
Demand correlates with traffic and amenities	Areas with high traffic and many nearby amenities are assumed to have proportionally higher EV charging demand.
Distance signals underservice	Areas further from existing chargers are treated as underserved and therefore higher priority, regardless of other factors.
SA2 boundaries are stable	The analysis uses 2021 ABS SA2 boundaries. Changes to boundaries in future ABS releases are not accounted for.
Cluster patterns are consistent	KMeans clusters are assumed to reflect meaningful demand/supply groupings across the state.

Future Recommendations

The following recommendations were drawn from the blockers and recommendations identified in the project progress documentation from T3, 2025.

- Deploy the API to a shared cloud environment, such as AWS or Azure, for team-wide access.
- Connect API outputs to the EVAT web app frontend for live map visualization.
- Align the output schema with other EVAT use cases such as demand forecasting to enable cross-subproblem integration.
- Refresh the underlying dataset periodically as new charger installations and traffic data become available.
- Add authentication (API keys) before any public-facing deployment.